Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Artificial Intelligence
Agentic AI (IA462), Semester VIII
U23AI052 Lab 8 Submission

Experiment 8

Game-Theoretic Intuition for Agent Interaction — Equilibria and Repeated Play

Aim

To analyse the interaction of two agents as a normal-form game by computing best responses, strictly dominated strategies and Nash equilibria for four standard games, and to show how repetition changes the outcome by running a tournament of history-dependent strategies in the iterated Prisoner's Dilemma.

Theory

In Experiment 7 the environment was indifferent to the agent. Here the payoff of an action depends on what another agent chooses at the same moment, so no action has a value until the other agent's behaviour is fixed. A normal-form game states the players, the strategies available to each and the payoff to both for every combination. A strategy is a best response if nothing else pays better against what the opponent is doing, it is strictly dominated if some other strategy pays better against everything the opponent might do, and a Nash equilibrium is a combination in which every agent is playing a best response, so no one gains by deviating alone. Equilibrium is a stability claim, not a claim about welfare: the Prisoner's Dilemma has exactly one equilibrium and both players prefer an outcome that is not it.

Equilibria need not be unique and need not exist in pure strategies: a coordination game has two, which is a different problem — agreeing which — and a strictly competitive game may have none until each side is allowed to randomise, at probabilities that leave the opponent indifferent. Repetition changes the analysis again. When the same agents meet repeatedly a strategy is a function of history rather than a single action, so an agent can answer defection with defection, and cooperation becomes sustainable provided the interaction is likely enough to continue. What makes this fragile in practice is noise: if an action is occasionally mis-implemented, a reciprocating pair echoes the error indefinitely.

Table 1. The four games to be analysed. Each cell gives (row payoff, column payoff).

Game	Strategies	Payoffs
Prisoner's Dilemma	C, D	(C,C) = (3, 3) (C,D) = (0, 5) (D,C) = (5, 0) (D,D) = (1, 1)
Stag Hunt	S, H	(S,S) = (4, 4) (S,H) = (0, 3) (H,S) = (3, 0) (H,H) = (2, 2)
Chicken	swerve, straight	(sw,sw) = (3, 3) (sw,st) = (1, 4) (st,sw) = (4, 1) (st,st) = (0, 0)
Matching Pennies	H, T	(H,H) = (1, −1) (H,T) = (−1, 1) (T,H) = (−1, 1) (T,T) = (1, −1)

Algorithm

Algorithm 1 — Equilibria of the one-shot games

1. Represent each game of Table 1 as the two strategy lists together with a table mapping a pair of strategies to a pair of payoffs. Write a function returning, for each player, the best responses to every strategy of the opponent, and a function returning the strictly dominated strategies of each player.
2. Report the pure-strategy Nash equilibria as the combinations in which both strategies are best responses to

each other, and where a mixed equilibrium exists, obtain it by solving for the probability that leaves the opponent indifferent between its two strategies. Print for every game its dominated strategies, its pure equilibria and its mixed equilibrium.

```
=== algo1: one-shot game analysis ===
prisoners dilemma dominated: (['C'], ['C']) pure equilibria: [('D', 'D')] mixed equilibrium: (0.6666666666
666666, 0.6666666666666666)
stag hunt dominated: ([], []) pure equilibria: [('H', 'H'), ('S', 'S')] mixed equilibrium: (0.666666666666
6666, 0.6666666666666666)
chicken dominated: ([], []) pure equilibria: [('sw', 'st'), ('st', 'sw')] mixed equilibrium: (0.5, 0.5)
matching pennies dominated: ([], []) pure equilibria: [] mixed equilibrium: (0.5, 0.5)
```

Algorithm 2 — Tournament in the iterated Prisoner's Dilemma

1. Implement each strategy as a function of the two histories so far, returning the next action: always cooperate, always defect, tit-for-tat, grim trigger, a generous tit-for-tat that forgives a defection with a stated probability, and a random player.
2. Play every ordered pair of strategies, including a strategy against itself, for a fixed number of rounds, averaging over several random seeds, and accumulate the payoff per round. Print the matrix of average payoffs, the mean each strategy obtains across all opponents, and the resulting ranking.

```
=== algo2: iterated prisoner dilemma tournament ===

tournament matrix
always cooperate [3.0, 0.0, 3.0, 3.0, 3.0, 1.464] mean 2.244
always defect [5.0, 1.0, 1.04, 1.04, 1.496, 2.952] mean 2.088
tit for tat [3.0, 0.99, 3.0, 3.0, 3.0, 2.202] mean 2.532
grim trigger [3.0, 0.99, 3.0, 3.0, 3.0, 2.918] mean 2.651
generous tit for tat [3.0, 0.876, 3.0, 3.0, 3.0, 2.19] mean 2.511
random [4.024, 0.512, 2.212, 0.588, 2.576, 2.322] mean 2.039
ranking: ['grim trigger', 'tit for tat', 'generous tit for tat', 'always cooperate', 'always defect', 'random']
```

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the program below, including for each game its dominated strategies and equilibria, and for the tournament the full matrix of average payoffs per round with the mean and ranking of every strategy.

Observations

Comment on why cooperation is strictly dominated in the Prisoner's Dilemma although both players prefer mutual cooperation to the equilibrium, on what having two equilibria rather than one means for agents that cannot talk to each other, on which strategy wins the tournament and whether winning its own pairings is the same as beating its opponents, and on what repetition supplies that a single encounter does not.

Exercises

1. Add noise: with a stated probability each action is mis-implemented as its opposite before the payoff is read. Re-run the tournament at noise 0, 0.01, 0.05 and 0.1, reporting the mean of every strategy and the payoff each of tit-for-tat, grim trigger and generous tit-for-tat obtains against a copy of itself. State which strategy degrades most and why, and which survives noise best.

```
=== ex1: tournament with implementation noise ===

noise sweep
0 {'always cooperate': 2.244, 'always defect': 2.088, 'tit for tat': 2.532, 'grim trigger': 2.651, 'genero
us tit for tat': 2.511, 'random': 2.039}
0.01 {'always cooperate': 2.134, 'always defect': 2.109, 'tit for tat': 2.291, 'grim trigger': 2.315, 'gen
erous tit for tat': 2.341, 'random': 2.039}
0.05 {'always cooperate': 1.798, 'always defect': 2.183, 'tit for tat': 2.121, 'grim trigger': 2.211, 'gen
erous tit for tat': 2.077, 'random': 2.042}
0.1 {'always cooperate': 1.787, 'always defect': 2.272, 'tit for tat': 2.057, 'grim trigger': 2.262, 'gene
rous tit for tat': 2.044, 'random': 2.045}
```

2. Replace the fixed number of rounds with a continuation probability w, so the expected payoff per round is the discounted average. Compute the normalised payoff of tit-for-tat against tit-for-tat and of always-defect against tit-for-tat for w from 0 to 0.9, find the value of w above which defecting no longer pays, and check it against the condition obtained by equating the two expressions algebraically.

```
=== ex2: continuation probability ===

continuation probability
0 tft/tft 3 defect/tft 5
0.2 tft/tft 3 defect/tft 4.2
0.4 tft/tft 3 defect/tft 3.4
0.6 tft/tft 3 defect/tft 2.6
0.8 tft/tft 3 defect/tft 1.7999999999999998
0.9 tft/tft 3 defect/tft 1.4
```